Smart Task Manager

App.js

```
import React, { useState, useEffect } from "react";
import "./App.css";

function App() {
  const [tasks, setTasks] = useState([]);
  const [input, setInput] = useState("");
  const [filter, setFilter] = useState("all");
  const [editIndex, setEditIndex] = useState(null);

  // Load tasks from localStorage
  useEffect(() => {
    const saved = JSON.parse(localStorage.getItem("tasks")) || [];
    setTasks(saved);
  }, []);

  // Save tasks to localStorage
  useEffect(() => {
    localStorage.setItem("tasks", JSON.stringify(tasks));
  }, [tasks]);

  // Add or update task
  const handleAddOrUpdate = () => {
    const trimmed = input.trim();
    if (!trimmed) return;

    if (editIndex !== null) {
      const newTasks = tasks.map((task, i) =>
        i === editIndex ? { ...task, text: trimmed } : task
```

```
    );
    setTasks(newTasks);
    setEditIndex(null);
  } else {
    setTasks([...tasks, { text: trimmed, completed: false }]);
  }
  setInput("");
};

const toggleTask = (index) => {
  setTasks(
    tasks.map((task, i) =>
      i === index ? { ...task, completed: !task.completed } : task
    )
  );
};

const deleteTask = (index) => {
  setTasks(tasks.filter((_, i) => i !== index));
};

const editTask = (index) => {
  setInput(tasks[index].text);
  setEditIndex(index);
};

const filteredTasks = tasks.filter((task) => {
  if (filter === "completed") return task.completed;
  if (filter === "pending") return !task.completed;
  return true;
```

```
});

const handleKeyPress = (e) => {
  if (e.key === "Enter") handleAddOrUpdate();
};

return (
  <div className="container">
    <h2>Task Manager</h2>

    <div className="input-group">
      <input
        value={input}
        onChange={(e) => setInput(e.target.value)}
        onKeyDown={handleKeyPress}
        placeholder="Enter a task"
      />
      <button onClick={handleAddOrUpdate}>
        {editIndex !== null ? "Update" : "Add"}
      </button>
    </div>

    <div className="filters">
      <button onClick={() => setFilter("all")}>All</button>
      <button onClick={() => setFilter("completed")}>Completed</button>
      <button onClick={() => setFilter("pending")}>Pending</button>
    </div>

    <ul>
      {filteredTasks.length === 0 && <p>No tasks to show!</p>}
```

```
        {filteredTasks.map((task, index) => (
          <li key={index} className="task-item">
            <span
              onClick={() => toggleTask(tasks.indexOf(task))}
              className={task.completed ? "completed" : ""}
            >
              {task.text}
            </span>
            <div>
              <button onClick={() => editTask(tasks.indexOf(task))}>Edit</button>
              <button onClick={() => deleteTask(tasks.indexOf(task))}>X</button>
            </div>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

App.css

```
/* Container */
body {
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
  background: linear-gradient(135deg, #ff9a9e 0%, #fad0c4 50%, #fbc2eb 100%);
  min-height: 100vh;
  display: flex;
```

```
  justify-content: center;
  align-items: center;
}

/* App Box */
.container {
  width: 400px;
  background: rgba(255, 255, 255, 0.9);
  padding: 25px;
  border-radius: 20px;
  box-shadow: 0 8px 25px rgba(0,0,0,0.2);
  backdrop-filter: blur(10px);
  text-align: center;
}

/* Heading */
h2 {
  margin-bottom: 20px;
  color: #6a1b9a; /* purple shade */
  text-shadow: 1px 1px 3px rgba(0,0,0,0.2);
}

/* Input and button */
.input-group {
  display: flex;
  margin-bottom: 20px;
}

input {
  flex: 1;
```

```
  padding: 10px;
  border: 2px solid #d1c4e9;
  border-radius: 12px;
  font-size: 16px;
  outline: none;
}

input:focus {
  border-color: #7e57c2;
  box-shadow: 0 0 8px #7e57c2;
}

button {
  padding: 10px 16px;
  margin-left: 5px;
  border: none;
  border-radius: 12px;
  background: linear-gradient(45deg, #ff6b6b, #f06595);
  color: white;
  font-weight: bold;
  cursor: pointer;
  transition: transform 0.2s;
}

button:hover {
  transform: scale(1.1);
}

/* Filters */
.filters {
```

```
  margin-bottom: 15px;
}

.filters button {
  margin: 0 3px;
  padding: 6px 12px;
  border-radius: 12px;
  border: none;
  font-weight: bold;
  cursor: pointer;
  transition: 0.2s;
}

.filters button:nth-child(1) { background-color: #2196f3; color: white; }
.filters button:nth-child(2) { background-color: #4caf50; color: white; }
.filters button:nth-child(3) { background-color: #ff9800; color: white; }

.filters button:hover {
  opacity: 0.85;
}

/* Task List */
ul {
  padding: 0;
}

li.task-item {
  list-style: none;
  display: flex;
  justify-content: space-between;
```

```
  align-items: center;
  margin-bottom: 10px;
  padding: 10px 12px;
  border-radius: 15px;
  background: linear-gradient(135deg, #e1bee7, #f3e5f5);
  box-shadow: 0 4px 12px rgba(0,0,0,0.1);
}

li.task-item span {
  flex-grow: 1;
  text-align: left;
  cursor: pointer;
  font-size: 16px;
  transition: 0.2s;
}

li.task-item span.completed {
  text-decoration: line-through;
  color: gray;
  opacity: 0.6;
}

/* Edit/Delete Buttons */
li.task-item div button {
  margin-left: 5px;
  padding: 5px 10px;
  border-radius: 12px;
  border: none;
  cursor: pointer;
  font-weight: bold;
```

```
  transition: 0.2s;
}

li.task-item div button:first-child {
  background: #00bcd4; /* Edit button cyan */
  color: white;
}

li.task-item div button:first-child:hover {
  background: #0097a7;
}

li.task-item div button:last-child {
  background: #ff5252; /* Delete button red */
  color: white;
}

li.task-item div button:last-child:hover {
  background: #e53935;
}
```